AULA 03
ESTRUTURAS DE CONTROLE

Programação em Python • Parte 1 de 2
Curso de Capacitação em Desenvolvimento Full Stack
Tema: Estruturas de Controle • 8 horas
Professor: Msc. Hygo Sousa de Oliveira

(95) 99173-5676 / (92) 99171-0391
www.iteam.org.br
Av. Glaycon de Paiva nº 1820,
Mecejana
CEP: 69.304-560
Boa Vista/RR
CNPJ: 48.653.884/0001-00

Roteiro da aula

O que vamos ver nesta aula

1. Estruturas condicionais: if, elif e else
2. Condições compostas e operador ternário
3. Estruturas de repetição: for e while
4. Controle de fluxo: break, continue e pass
5. Laços aninhados e erros comuns
6. Exemplos guiados para fazer em sala

Objetivos de aprendizagem

Ao final da aula você será capaz de:

- **Tomar decisões com if, elif e else**
  Condições simples e compostas
- **Repetir tarefas com for e while**
  range(), coleções e laços com condição de parada
- **Controlar laços com break, continue e pass**
  Interromper, pular e deixar blocos em branco
- **Reconhecer e evitar erros comuns**
  Indentação, = versus ==, laços infinitos

Módulo 2 – Estruturas de Controle

- Estruturas condicionais: if, elif, else
- Estruturas de repetição: for, while
- Controle de fluxo: break, continue, pass
- Tratamento de exceções: try, except, finally

if: a decisão mais simples

O bloco só executa quando a condição é verdadeira

decisao.py

```
idade = 20
if idade >= 18:
    print("Maior de idade")
print("Fim do programa")
```

SAÍDA

```
Maior de idade
Fim do programa
```

ATENÇÃO Dois-pontos `:` no fim do `if` e indentação de 4 espaços no bloco são obrigatórios.

if / else

Dois caminhos possíveis

compra.py

```
saldo = 120.0
valor = 150.0
if valor <= saldo:
    saldo -= valor
    print("Compra aprovada")
else:
    print("Saldo insuficiente")
print(f"Saldo: R$ {saldo:.2f}")
```

SAÍDA

```
Saldo insuficiente
Saldo: R$ 120.00
```

DICA O `else` não tem condição: ele cobre tudo o que o `if` não cobriu.

if / elif / else

Várias faixas de decisão, apenas um bloco executa

votar.py

```
idade = int(input("Digite sua idade: "))
if idade >= 18:
    print("Você é maior de idade.")
elif idade >= 16:
    print("Menor de idade, mas pode votar.")
else:
    print("Menor de idade e não pode votar.")
```

SAÍDA

```
Digite sua idade: 17
Menor de idade, mas pode votar.
```

EM SALA O Python testa de cima para baixo e executa **apenas o primeiro** bloco verdadeiro. Teste com 20, 17 e 12.

Condições compostas

Combine comparações com and, or e not

A	B	A and B	A or B
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

intervalo.py

```
numero = 15
if numero >= 10 and numero <= 20:
    print("Número válido.")
else:
    print("Número inválido.")
print(10 <= numero <= 20)
```

SAÍDA

```
Número válido.
True
```

DICA `not` inverte o valor: `not True` é `False`. Comparações encadeadas como `10 <= x <= 20` são válidas.

Valores verdadeiros e operador ternário

Todo valor do Python pode ser usado como condição

verdade.py

```
nome = ""
print(bool(nome), bool([]), bool(0), bool("a"))
if not nome:
    print("Nome vazio")
idade = 20
status = "adulto" if idade >= 18 else "menor"
print(status)
```

SAÍDA

```
False False False True
Nome vazio
adulto
```

NOTA São "falsos": `0`, `0.0`, `""`, `[]`, `{}`, `None` e `False`. O restante é verdadeiro.

Exemplo guiado 1: ingresso do cinema

Combine if, elif, else e operadores lógicos

DESAFIO Menor de 12 ou 60+: R$ 10. Estudante: R$ 15. Demais: R$ 30.

ingresso.py

```
idade = int(input("Idade: "))
est = input("Estudante (s/n)? ") == "s"
if idade < 12 or idade >= 60:
    preco = 10
elif est:
    preco = 15
else:
    preco = 30
print(f"Ingresso: R$ {preco:.2f}")
```

SAÍDA

```
Idade: 25
Estudante (s/n)? s
Ingresso: R$ 15.00
```

Repetição com for e range()

Percorrendo uma sequência numérica

for_range.py

```
for numero in range(1, 6):
    quadrado = numero ** 2
    print(f"O quadrado de {numero} é {quadrado}")
```

SAÍDA

```
O quadrado de 1 é 1
O quadrado de 2 é 4
O quadrado de 3 é 9
O quadrado de 4 é 16
O quadrado de 5 é 25
```

DICA `range(inicio, fim, passo)`: o fim é exclusivo. `range(5)` vai de 0 a 4; `range(10, 0, -2)` gera 10, 8, 6, 4, 2.

for sobre coleções

Listas, textos e enumerate()

for_colecoes.py

```
frutas = ["maçã", "banana", "uva"]
for fruta in frutas:
    print(fruta.upper())
for i, fruta in enumerate(frutas, start=1):
    print(i, fruta)
for letra in "Ana":
    print(letra, end="-")
```

SAÍDA

```
MAÇÃ
BANANA
UVA
1 maçã
2 banana
3 uva
A-n-a-
```

NOTA O `for` percorre qualquer sequência. `enumerate()` entrega o índice junto com o valor.

while: repetir enquanto for verdadeiro

Ideal quando não sabemos quantas repetições serão necessárias

while.py

```
contador = 0
while contador < 5:
    print(f"Iteração: {contador}")
    contador += 1
print("Fim do laço")
```

SAÍDA

```
Iteração: 0
Iteração: 1
Iteração: 2
Iteração: 3
Iteração: 4
Fim do laço
```

ATENÇÃO Esqueceu o incremento? O laço fica **infinito**. Use Ctrl+C para interromper.

while com valor sentinela

Ler dados até o usuário avisar que terminou

sentinela.py

```
soma = 0
numero = int(input("Número (0 encerra): "))
while numero != 0:
    soma += numero
    numero = int(input("Número (0 encerra): "))
print(f"Soma total: {soma}")
```

SAÍDA

```
Número (0 encerra): 5
Número (0 encerra): 10
Número (0 encerra): 3
Número (0 encerra): 0
Soma total: 18
```

DICA O 0 é o valor **sentinela**: ele sinaliza o fim da leitura e não entra na soma.

break, continue e pass

Alterando o comportamento padrão dos laços

Instrução	Função	Exemplo prático
`break`	Interrompe o laço imediatamente e sai dele	Busca por um item e para ao encontrar
`continue`	Pula para a próxima iteração	Ignora números pares em uma lista
`pass`	Instrução nula; marcador de lugar sintático	Estruturas vazias em desenvolvimento

pass.py

```
def funcao_em_desenvolvimento():
    pass   # evita erro de sintaxe enquanto a lógica não existe
```

break e continue em ação

Dois laços, dois comportamentos

fluxo.py

```
for i in range(10):
    if i == 5:
        break
    print(i, end=" ")
print()
for i in range(6):
    if i % 2 == 0:
        continue
    print(i, end=" ")
```

SAÍDA

```
0 1 2 3 4
1 3 5
```

NOTA `break` sai do laço inteiro; `continue` abandona só a repetição atual.

Exemplo guiado 2: contador de vogais

for + if + operador in

DESAFIO Leia uma frase e conte quantas vogais (a, e, i, o, u) ela possui.

vogais.py

```
frase = input("Frase: ").lower()
vogais = 0
for letra in frase:
    if letra in "aeiou":
        vogais += 1
print(f"Vogais: {vogais}")
```

SAÍDA

```
Frase: Programação em Python
Vogais: 6
```

EM SALA Passo a passo: 1) normalize com `.lower()`; 2) percorra os caracteres; 3) teste com `in`; 4) acumule.

Exemplo guiado 3: laços aninhados

Um laço dentro de outro: tabuada em formato de tabela

tabela.py

```
for i in range(1, 5):
    for j in range(1, 5):
        print(f"{i * j:3}", end="")
    print()
```

SAÍDA

```
  1  2  3  4
  2  4  6  8
  3  6  9 12
  4  8 12 16
```

NOTA O laço interno completa todas as voltas a cada volta do laço externo. `:3` reserva 3 posições para alinhar.

Erros comuns em controle de fluxo

Reconheça o problema antes de depurar

1 = em vez de ==

`if x = 5:` gera SyntaxError. Para comparar, use `==`.

2 Laço infinito

Condição do `while` que nunca fica falsa, por exemplo sem incremento.

3 Indentação errada

Uma linha fora do recuo muda o significado do programa.

4 Erro de um a mais ou a menos

`range(1, 5)` vai até 4. Para chegar a 5, use `range(1, 6)`.

Atividades da Aula 3

Módulo 2: condicionais e repetição

- **Ex. 01 a 08 – Estruturas condicionais e de repetição**
  Classificador de temperatura, validador de acesso, tabuada interativa, contador regressivo, busca com `break` e filtro com `continue`

DICA Revise `if/elif/else`, `for`, `while`, `break` e `continue` antes de começar os exercícios.

GIT Enunciados, instruções de execução e de entrega: **github.com/Hygo/python-fullstack-iteam-2sem**

Obrigado(a)!

Próxima aula: Tratamento de exceções e fechamento do Módulo 2
ITEAM – Instituto Tecnológico Educacional da Amazônia